Assignment 2 — Solution

Weeks 3 & 4 | SQL, Normalization & Design Justification

Part I: Short Answer / Reasoning Questions

Q1 — NULL Comparisons and Aggregates

Why the original query always returns 0:

In SQL, NULL represents an unknown value. Any comparison involving NULL — including `= NULL` — evaluates to UNKNOWN, not TRUE. Because the WHERE clause only keeps rows where the condition is TRUE, every row is filtered out, and COUNT(*) returns 0.

Corrected query:

```
SELECT COUNT(*)
FROM Student
WHERE phone_number IS NULL;
```

Why NULL is excluded from most aggregates:

SQL treats NULL as the absence of a known value. Including it in arithmetic or comparisons would produce undefined or misleading results. Therefore, aggregate functions (SUM, AVG, COUNT(col), MIN, MAX) silently skip NULL values, and comparison operators such as =, <, > never yield TRUE for a NULL operand — only IS NULL / IS NOT NULL can test for it directly.

Q2 — DISTINCT Inside GROUP BY

The student is **correct** — the DISTINCT keyword is redundant here.

When GROUP BY department is present, the result set already has exactly one row per unique department value. DISTINCT would attempt to remove duplicate rows from the output, but since (department, COUNT(*)) pairs are inherently unique per group, it has nothing to deduplicate. The query produces identical results with or without DISTINCT, making its inclusion misleading and unnecessary.

Q3 — INNER JOIN vs LEFT JOIN Row Count Difference

The difference in row count tells us that **some customers have never placed an order** — i.e., there are rows in the Customer table with no matching row in the Order table.

An INNER JOIN only returns rows where a match exists in both tables, so unmatched customers are silently dropped. A LEFT JOIN (with Customer as the left table) retains every customer row, filling Order columns with NULL when no match exists. The extra rows returned by LEFT JOIN correspond exactly to customers who have placed zero orders.

Q4 — Invalid SELECT with GROUP BY

The query is invalid because `employee_name` is neither in the GROUP BY clause nor wrapped in an aggregate function. In standard SQL, every column in SELECT must either appear in GROUP BY or be aggregated.

Conceptually, if SQL were to 'allow' this, it would need to pick a single `employee_name` to display for each department group — but multiple employees belong to each department. There is no principled way to choose one; any choice would be arbitrary and misleading. SQL therefore prohibits it.

Corrected query (showing department and average salary only):

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

If a specific employee name is needed per group, a window function or subquery approach must be used:

```
-- Example: also show one employee per dept using a subquery
SELECT department, employee_name, AVG(salary) OVER (PARTITION BY department)
FROM Employee;
```

Q5 — WHERE vs HAVING

Error: AVG(salary) is an aggregate function, and aggregate functions cannot appear in the WHERE clause.

Why: SQL processes clauses in this logical order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. The WHERE clause filters individual rows *before* grouping occurs, so aggregated values do not yet exist at that stage. HAVING is evaluated after GROUP BY and is specifically designed to filter groups based on aggregate results.

Corrected query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6 — Correlated vs Non-Correlated Subqueries

(a) Products priced above the overall average — Non-correlated subquery

```
SELECT *
FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

A non-correlated subquery is appropriate here because the threshold (overall average) is a single value computed once across the entire Product table, independent of any particular row being evaluated in the outer query.

(b) Employees earning more than their own department's average — Correlated subquery

```
SELECT e.employee_id, e.name, e.salary, e.department
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.department = e.department  -- references outer row
);
```

A correlated subquery is appropriate because the comparison value (department average) depends on the department of the current outer row. The inner query must re-execute for each row in the outer query.

Performance implication:

A non-correlated subquery executes exactly once; its result is computed and reused for every row in the outer query. A correlated subquery re-executes for every row in the outer table — if the outer table has N rows, the subquery runs N times. For large tables this can be orders of magnitude slower. In practice, correlated subquery patterns are often rewritten as JOINs with aggregates, or optimised by the query planner using materialisation.

Part II: Long Answer — Online Food Delivery Platform

Schema reminder:

```
Customer(customer_id, name, email, phone)
Restaurant(restaurant_id, name, cuisine_type, rating)
MenuItem(item_id, restaurant_id, item_name, price)
Order(order_id, customer_id, restaurant_id, order_date, status)
OrderItem(order_id, item_id, quantity)
```

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name, c.email
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Justification: A LEFT JOIN retains all Customer rows regardless of whether a matching Order exists. Filtering on o.order_id IS NULL then isolates exactly those customers with no matching order. An INNER JOIN would silently drop these customers, making it unable to capture the 'never ordered' condition. An alternative using NOT EXISTS is equally correct.

(b) Average order value per restaurant (more than 5 orders)

```
SELECT o.restaurant_id,
       AVG(order_total) AS avg_order_value
FROM (
    SELECT oi.order_id,
           SUM(mi.price * oi.quantity) AS order_total
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY oi.order_id
) AS order_values
JOIN "Order" o ON order_values.order_id = o.order_id
GROUP BY o.restaurant_id
HAVING COUNT(*) > 5;
```

Justification: A subquery first computes the total value of each individual order (SUM of price x quantity). The outer query then joins back to Order to obtain the restaurant, groups by restaurant, and uses HAVING (not WHERE) to filter groups with more than 5 orders — HAVING is required because COUNT(*) is an aggregate that only exists after grouping.

(c) Restaurants whose average order value exceeds the platform-wide average

```
-- CTE version (equivalent to nested subquery)
WITH RestaurantAvg AS (
    SELECT o.restaurant_id,
           AVG(order_total) AS avg_order_value
    FROM (
        SELECT oi.order_id,
               SUM(mi.price * oi.quantity) AS order_total
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) AS order_values
    JOIN "Order" o ON order_values.order_id = o.order_id
    GROUP BY o.restaurant_id
    HAVING COUNT(*) > 5
)
SELECT restaurant_id, avg_order_value
FROM RestaurantAvg
WHERE avg_order_value > (SELECT AVG(avg_order_value) FROM RestaurantAvg);
```

Justification: A non-correlated subquery is used for the platform-wide average because that value is a single scalar computed once from RestaurantAvg — it does not depend on the current restaurant row being evaluated. A correlated subquery would be unnecessary and wasteful here since the comparison value is the same for every row. The CTE (or equivalent nested subquery) avoids rewriting the per-restaurant aggregation logic twice.

(d) Most ordered menu item (by total quantity) for each restaurant

```
SELECT r.restaurant_id, r.item_id, r.item_name, r.total_qty
```

```
FROM (
    SELECT mi.restaurant_id,
           mi.item_id,
           mi.item_name,
           SUM(oi.quantity) AS total_qty,
           RANK() OVER (
               PARTITION BY mi.restaurant_id
               ORDER BY SUM(oi.quantity) DESC
           ) AS rnk
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) AS r
WHERE r.rnk = 1;
```

Justification: A window function RANK() OVER (PARTITION BY restaurant_id ORDER BY total_qty DESC) correctly handles the 'top per group' problem. The outer WHERE rnk = 1 retains only the top item(s) per restaurant.

Limitation of basic GROUP BY + MAX approaches:

A naive `GROUP BY restaurant_id HAVING total_qty = MAX(total_qty)` pattern cannot directly correlate the maximum quantity back to the specific item that produced it. Workarounds (such as joining back on the max value) return multiple rows if two items share the same maximum quantity — a tie — with no way to break or acknowledge it. Window functions like RANK() or DENSE_RANK() handle ties explicitly and scale more cleanly to top-N (not just top-1) requirements.

Part III: Normalization

Table under analysis:

```
Enrollment(student_id, student_name, course_id, course_name,
           instructor_id, instructor_name, instructor_office,
           grade, enrollment_date)
```

Given constraints: A course has exactly one instructor. An instructor has exactly one office. (student_id, course_id) is the composite primary key.

Q1 — Functional Dependencies

```
FD1:  student_id          --> student_name
FD2:  course_id           --> course_name
FD3:  course_id           --> instructor_id
FD4:  instructor_id       --> instructor_name
FD5:  instructor_id       --> instructor_office
FD6:  (student_id, course_id) --> grade
FD7:  (student_id, course_id) --> enrollment_date
```

FD6 and FD7 are the only dependencies that require the full composite key; all others depend on only part of the key (FD1, FD2, FD3) or on a non-key attribute (FD4, FD5).

Q2 — First Normal Form (1NF)

Yes, the table is in **1NF**. All attributes hold atomic (single-valued, indivisible) values — there are no repeating groups or multi-valued attributes. Every row is uniquely identified by the composite key (student_id, course_id). The relation therefore satisfies the requirements of 1NF.

Q3 — Partial Dependency Violating 2NF

The table is **not in 2NF**. Several non-key attributes depend on only *part* of the composite key:

```
student_id  --> student_name     (partial: only needs student_id)
course_id   --> course_name       (partial: only needs course_id)
course_id   --> instructor_id     (partial: only needs course_id)
```

Anomaly example — student_name partial dependency:

Suppose a student changes their name. Because student_name appears in every enrollment row for that student, every such row must be updated individually. If even one row is missed, the database becomes inconsistent — one enrollment shows the old name, another shows the new name for the same student. This is an *update anomaly*.

An *insertion anomaly* also arises: a new student cannot be recorded until they enrol in at least one course, because student_name has no home outside the Enrollment table.

Q4 — Transitive Dependency Violating 3NF

Even after resolving 2NF violations, transitive dependencies remain:

```
course_id --> instructor_id --> instructor_name
course_id --> instructor_id --> instructor_office
```

Here instructor_name and instructor_office depend on instructor_id, which is itself a non-key attribute (in a 2NF-decomposed Course table). This is a transitive dependency through a non-key attribute, violating 3NF.

Anomaly: If an instructor moves offices, every Course row listing that instructor must be updated. If an instructor teaches no courses currently, their office information cannot be stored at all — an insertion anomaly.

Q5 — 3NF Decomposition

```
Student(<u>student_id</u>, student_name)
    -- Justified by FD1: student_id --> student_name

Course(<u>course_id</u>, course_name, instructor_id)
```

```
    -- Justified by FD2 + FD3: course_id --> course_name, instructor_id

Instructor(<u>instructor_id</u>, instructor_name, instructor_office)
    -- Justified by FD4 + FD5: instructor_id --> instructor_name, instructor_office

Enrollment(<u>student_id</u>, <u>course_id</u>, grade, enrollment_date)
    -- Justified by FD6 + FD7: (student_id, course_id) --> grade, enrollment_date
    -- FK: student_id references Student; course_id references Course
```

All four tables are in 3NF: every non-key attribute depends on the whole key and nothing but the key. All original data can be reconstructed by joining these tables, preserving lossless decomposition.

Part IV: Design Justification (~200 words)

JOIN vs Subquery choice (Part II, question a):

For the 'customers who never placed an order' query, I could have written it using either a LEFT JOIN with IS NULL or a NOT EXISTS correlated subquery. I chose the LEFT JOIN approach because it is more readable for most SQL practitioners — the pattern 'left-join then filter on NULL foreign key' is idiomatic and immediately signals the intent. A NOT EXISTS subquery is logically equivalent and sometimes preferred when NULL semantics of the join column could be ambiguous, but here customer_id is a primary key and cannot itself be NULL, so the LEFT JOIN approach is both safe and clear. Performance-wise, modern query optimisers typically produce the same execution plan for both forms on indexed keys.

Normalisation trade-off (Part III):

Decomposing Enrollment into four 3NF tables eliminates anomalies but adds query complexity. Any report that previously read from one table — such as 'list all students with their course names and instructor offices' — now requires three or four joins. This increases query length, the risk of join mistakes, and potentially query latency when tables are large and indexes are absent. Real systems therefore sometimes intentionally denormalise: a data warehouse might keep a wide 'fact' table with redundant name columns to avoid expensive joins at reporting time, accepting the trade-off that some update anomalies are managed through ETL processes rather than schema constraints.

End of Assignment 2 Solution